Examen Práctico Complementario: Programación Python

Bitácora de Interacción con Inteligencia Artificial

Andrea Arroyo Escalante
Matrícula: 35163832

Materia: Computación Aplicada
Ciclo Escolar: Semestre Agosto-Diciembre 2026
Catedrático: Carlos Héctor Castañeda Ramírez

30 de octubre de 2026

Documento de evidencia del proceso de interacción con herramientas de Inteligencia Artificial.

1. Introducción

Este documento pertence a la parte práctica del Segundo Examen Parcial de la materia Computación Aplicada del semestre Agosto-Diciembre 2026 de la Universidad Autónoma de Zacatecas. Aquí se muestra paso a paso la interacción que se tuvo con la inteligencia artificial de *Google Gemini* para resolver el problema práctico planteado en el examen que que indica crear un *"AeroRegistro (p136-segundo-examen-parcial.py) empleando ingeniería de prompts gradual con Inteligencia Artificial."*. Aquí se muestra la evidencia de las cinco fases.

2. AeroRegistro

2.1 Planteamiento del problema

Se desea procesar el registro de vuelos de una aerolínea (*"AeroRegistro"*). Para ello, deberá solicitar al usuario los siguientes datos por cada vuelo:

- numero_vuelo (identificador del vuelo, ej. 'AA-101')
- origen (ciudad de salida, ej. 'Ciudad de México')
- destino (ciudad de llegada, ej. 'Guadalajara')
- aerolinea (nombre de la aerolínea, ej. 'AeroMéxico', 'Volaris', 'Interjet')
- pasajeros (número de pasajeros a bordo, ej. 180)
- tarifa (precio del boleto en pesos, ej. 2350.75)

Los datos se deben almacenar en una lista, donde cada elemento de la lista sea un diccionario con los datos de un vuelo.

Requisitos técnicos obligatorios

- **Diccionarios:** Los datos de cada vuelo capturado deben almacenarse en un diccionario.
- **Listas:** Todos los diccionarios de vuelos se agregarán a una lista principal que representará el inventario.
- **Conjuntos (Sets):** Extraer un conjunto con los destinos únicos para garantizar que no haya ciudades repetidas.
- **Tuplas:** Generar una lista de tuplas para un "registro histórico inmutable", donde cada tupla contenga exclusivamente (*numero_vuelo*, *aerolinea*, *destino*).

2.2 Requisitos de salida

Al terminar la captura (ej. cuando el usuario ingrese un número de vuelo vacío o "*"), el programa deberá procesar las colecciones e imprimir:

- **Datos Crudos:**La lista de diccionarios tal como se almacenó.
- **Formato Tabular:** Los datos de los vuelos organizados en una tabla clara.
- **Destinos Únicos:** El conjunto (set) resultante de ciudades de destino.
- **Registro Inmutable:** La lista de tuplas generada.

Resumen del registro:

- Total de vuelos registrados.
- Cuántos vuelos por aerolínea.
- Cuántos vuelos por ciudad de destino.
- La suma total de pasajeros y el promedio de pasajeros por vuelo.
- La suma total de tarifas (considerando una sola tarifa por vuelo) y la tarifa promedio.
- El número de vuelo y la tarifa del vuelo más caro y del más barato.

3. Guía de Ejecución Gradual con IA

Orden en el que se interactuó con un modelo de lenguaje (LLM), en este caso fue *Google Gemini*, dividida en cinco fases.

FASE 1: Estructura y Captura

> *"Solicitar a la IA la lógica del ciclo while para capturar las entradas del usuario y almacenarlas correctamente en una **Lista de Diccionarios**."*

A continuación se muestra la interacción con *Google Gemini* en forma de texto para la primera parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo A se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT 1.1: *"Actúa como un experto en Python. Empieza a desarrollar un programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro. Solamente escribe la lógica (estructura básica) del ciclo while para capturar las entradas del usuario, a quien se le van a solicitar los siguientes datos: numero_vuelo (identificador del vuelo, ej. 'AA-101'), origen (ciudad de salida, ej. 'Ciudad de México'), destino (ciudad de llegada, ej. 'Guadalajara'), aerolinea (nombre de la aerolínea, ej. 'AeroMéxico', 'Volaris', 'Interjet'), pasajeros (número de pasajeros a bordo, ej. 180) y tarifa (precio del boleto en pesos, ej. 2350.75). La entrada de datos termina cuando el usuario ingresa el caracter *. Los datos de cada vuelo ingresado deben almacenarse en un diccionario y todos los diccionarios de vuelos se agregarán a una lista principal que representará el inventario (lista de diccionarios)."*

RESPUESTA:

```
Python
vuelos = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' en el número de vuelo para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida
    if numero_vuelo == '*':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Conversión a entero y flotante para valores numéricos
    pasajeros = int(input("Número de pasajeros (ej. 180): "))
    tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))

    # Creación del diccionario por vuelo
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
```

```
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    # Agregar diccionario a la lista principal
    vuelos.append(vuelo)
    print("-> Vuelo guardado correctamente.\n")

print("\nCaptura finalizada. Inventario registrado:")
print(vuelos)
```

PROMPT 1.2: *"Modifica el nombre de la lista, actualmente se llama vuelos pero debe llamarse inventario. Agrega validación para los datos que lo requieren (pasajeros y tarifa, que además deben ser valores positivos). Agrega que la entrada, además de terminar al ingresar el caracter *, pueda terminar también cuando se ingresa un vuelo vacío."*

RESPUESTA:

```
Python
inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positivo.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

print("\nCaptura finalizada. Inventario registrado:")
print(inventario)
```

FASE 2: Extracción de Colecciones Especiales

> *"Instruir a la IA para recorrer los datos capturados y generar un Conjunto (Set) para los destinos únicos y una Lista de Tuplas para el registro inmutable."*

A continuación se muestra las interaccione con *Google Gemini* en forma de texto para la segunda parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo B se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT 2.1: *"Continuando con el desarrollo del programa para procesar el registro de vuelos de una*

aerolínea llamada AeroRegistro, y tomando en cuenta el código anterior. Ahora, recorre los datos capturados por el usuario, almacenados en inventario, y extrae un conjunto (set), llamado destinos_unicos, con los destinos únicos para garantizar que no haya ciudades repetidas. Además, genera una lista de tuplas, llamada historial, para un registro histórico inmutable, donde cada tupla contenga exclusivamente (numero_vuelo, aerolinea, destino). Agrega esto al código anterior, sin modificar lo que ya está."

RESPUESTA:

```
Python
inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positivo.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

print("\nCaptura finalizada. Inventario registrado:")
print(inventario)

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# Mostrar resultados
print("\n--- Resultados del Procesamiento ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
print(historial)
```

FASE 3: Formateo y Visualización

> *"Pedir la implementación de las salidas en pantalla: impresión de los datos crudos, visualización de tuplas/sets y el formateo de una tabla tabular alineada para los vuelos."*

A continuación se muestran las interaccione con *Google Gemini* en forma de texto para la tercera parte,

señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo C se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT 3.1: *"Continuando con el desarrollo del programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro, y tomando en cuenta el código anterior, procede a la parte la implementación de las salidas en pantalla: impresión de los datos crudos, visualización de tuplas/sets y el formateo de una tabla tabular alineada para los vuelos. Agrega esto al código anterior, sin modificar lo que ya está. Marca claramente el inicio y fin de la sección de entrada, y haz lo mismo para el inicio de salida."*

RESPUESTA:

```
Python
# ================================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# ================================================================================

inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positivo.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# ================================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ================================================================================


# ================================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# ================================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión de datos crudos
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
```

```
print(inventario)

# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
for item in historial:
    print(f"  Vuelo: {item[0]} | Aerolínea: {item[1]} | Destino: {item[2]}")

# 3. Formateo de tabla tabular alineada
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    # Encabezados de la tabla con f-strings y formato de ancho de columna
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15} |  {'PASAJEROS':<10} | {'TARIFA (MXN)':>12}"
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    # Filas de la tabla alineadas
    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# ==============================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ==============================================================================
```

FASE 4: Analítica y Cálculos Matemáticos

> *"Guiar a la IA para agregar contadores, sumar totales (pasajeros, tarifas), calcular promedios y encontrar los valores máximos y mínimos de los registros."*

A continuación se muestra la interacción con *Google Gemini* en forma de texto para la cuarta parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo D se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT 4.1: *"Continuando con el desarrollo del programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro, y tomando en cuenta el código anterior, procede a la parte la analítica y de cálculos matemáticos, para agregar contadores (calcular el número total de vuelos registrados, la cantidad de aerolíneas diferentes y el número de destinos únicos.), sumar totales (calcular el total de pasajeros registrados y la suma de todas las tarifas de los vuelos), calcular promedios (obtener el promedio de pasajeros por vuelo y el promedio de las tarifas registradas) y encontrar los valores máximos y mínimos de los registros (identificar el mayor y el menor número de pasajeros registrados en un vuelo, así como la tarifa más alta y la más baja). Agrega esto al código anterior, sin modificar lo que ya está. Muestra en salida esta información, además agrega un apartado de cálculos."*

RESPUESTA:

```
Python
# ==============================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# ==============================================================================

inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
```

```
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positivo.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# 3. Cálculos matemáticos y analítica
total_vuelos = len(inventario)

if total_vuelos > 0:
    # Contadores
    aerolineas_unicas = len({vuelo['aerolinea'] for vuelo in inventario})
    num_destinos_unicos = len(destinos_unicos)

    # Totales
    total_pasajeros = sum(vuelo['pasajeros'] for vuelo in inventario)
    total_tarifas = sum(vuelo['tarifa'] for vuelo in inventario)

    # Promedios
    promedio_pasajeros = total_pasajeros / total_vuelos
    promedio_tarifa = total_tarifas / total_vuelos

    # Máximos y Mínimos
    max_pasajeros = max(vuelo['pasajeros'] for vuelo in inventario)
    min_pasajeros = min(vuelo['pasajeros'] for vuelo in inventario)
    max_tarifa = max(vuelo['tarifa'] for vuelo in inventario)
    min_tarifa = min(vuelo['tarifa'] for vuelo in inventario)

# ==============================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ==============================================================================


# ==============================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# ==============================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión de datos crudos
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
for item in historial:
    print(f"  Vuelo: {item[0]} | Aerolínea: {item[1]} | Destino: {item[2]}")

# 3. Formateo de tabla tabular alineada
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    # Encabezados de la tabla con f-strings y formato de ancho de columna
```

```
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15} | {'PASAJEROS':<10} | {'TARIFA (MXN)':>12}"
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    # Filas de la tabla alineadas
    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# 4. Apartado de Cálculos y Analítica
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    print(f"• Total de vuelos registrados:      {total_vuelos}")
    print(f"• Cantidad de aerolíneas distintas:  {aerolineas_unicas}")
    print(f"• Número de destinos únicos:        {num_destinos_unicos}")
    print("-" * 50)
    print(f"• Total de pasajeros registrados:   {total_pasajeros:,} pasajeros")
    print(f"• Suma total de tarifas:            ${total_tarifas:,.2f} MXN")
    print("-" * 50)
    print(f"• Promedio de pasajeros por vuelo:  {promedio_pasajeros:.2f} pasajeros")
    print(f"• Promedio de tarifa por vuelo:     ${promedio_tarifa:,.2f} MXN")
    print("-" * 50)
    print(f"• Pasajeros por vuelo (Máx / Mín):  {max_pasajeros} / {min_pasajeros}")
    print(f"• Tarifa por vuelo (Máx / Mín):     ${max_tarifa:,.2f} MXN / ${min_tarifa:,.2f} MXN")

# ==============================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ==============================================================================
```

PROMPT 4.2: *"A partir de este código generado para AeroRegistro, para refinar el programa realiza algunas modificaciones. En Historial inmutable de vuelos (Lista de Tuplas), escribe solamente la lista historial. En las aerolíneas distintas, además de poner el total, haz un desglose mostrando el nombre de las aerolíneas y cuantas veces se repite cada una. Para el número de destinos únicos, además de mostrar el total, haz un desglose donde muestres el nombre del destino y cuántas veces se repite. El totales y promedios haz la siguiente modificación: que se imprima el total de pasajeros y debajo el promedio de pasajeros por vuelo. Luego muestra la línea separadora para después mostrar la suma total de tarifas y debajo el promedio de tarifa por vuelo. Para máximos y mínimos, escribe explícitamente los enunciados, por ejemplo VB-410 de $4,500.50 es el más caro, IJ-088 de $1,950.75 es el más barato. Lo mismo para pasajeros. Además, para un mejor acomodo visual en la salida, implementa que todos los valores de entrada se muestren en mayúsculas."*

RESPUESTA:

```
Python
# ==============================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# ==============================================================================

inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip().upper()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip().upper()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip().upper()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip().upper()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positivo.")
        except ValueError:
```

```
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# 3. Cálculos matemáticos y analítica
total_vuelos = len(inventario)

if total_vuelos > 0:
    # Desglose y conteo de aerolíneas
    conteo_aerolineas = {}
    for vuelo in inventario:
        aero = vuelo['aerolinea']
        conteo_aerolineas[aero] = conteo_aerolineas.get(aero, 0) + 1
    aerolineas_unicas = len(conteo_aerolineas)

    # Desglose y conteo de destinos
    conteo_destinos = {}
    for vuelo in inventario:
        dest = vuelo['destino']
        conteo_destinos[dest] = conteo_destinos.get(dest, 0) + 1
    num_destinos_unicos = len(conteo_destinos)

    # Totales y Promedios
    total_pasajeros = sum(vuelo['pasajeros'] for vuelo in inventario)
    promedio_pasajeros = total_pasajeros / total_vuelos

    total_tarifas = sum(vuelo['tarifa'] for vuelo in inventario)
    promedio_tarifa = total_tarifas / total_vuelos

    # Vuelos con valores Máximos y Mínimos
    vuelo_max_tarifa = max(inventario, key=lambda x: x['tarifa'])
    vuelo_min_tarifa = min(inventario, key=lambda x: x['tarifa'])

    vuelo_max_pasajeros = max(inventario, key=lambda x: x['pasajeros'])
    vuelo_min_pasajeros = min(inventario, key=lambda x: x['pasajeros'])

# ==============================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ==============================================================================


# ==============================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# ==============================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión de datos crudos
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
print(historial)

# 3. Formateo de tabla tabular alineada
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15} | {'PASAJEROS':<10} | {'TARIFA (MXN)':>12}"
    print("-" * len(header))
    print(header)
```

```
    print("-" * len(header))

    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# 4. Apartado de Cálculos y Analítica
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    print(f"• Total de vuelos registrados: {total_vuelos}\n")

    # Desglose de aerolíneas
    print(f"• Cantidad de aerolíneas distintas: {aerolineas_unicas}")
    print("  Desglose por aerolínea:")
    for aerolinea, cantidad in conteo_aerolineas.items():
        print(f"    - {aerolinea}: {cantidad} vuelo(s)")

    # Desglose de destinos
    print(f"\n• Número de destinos únicos: {num_destinos_unicos}")
    print("  Desglose por destino:")
    for destino, cantidad in conteo_destinos.items():
        print(f"    - {destino}: {cantidad} vuelo(s)")

    print("-" * 60)

    # Totales y Promedios agrupados
    print(f"• Total de pasajeros registrados:   {total_pasajeros:,} pasajeros")
    print(f"• Promedio de pasajeros por vuelo:  {promedio_pasajeros:.2f} pasajeros")
    print("-" * 60)
    print(f"• Suma total de tarifas:            ${total_tarifas:,.2f} MXN")
    print(f"• Promedio de tarifa por vuelo:     ${promedio_tarifa:,.2f} MXN")
    print("-" * 60)

    # Enunciados explícitos de Máximos y Mínimos
    print("• Análisis de Tarifas (Máximo y Mínimo):")
    print(f"  {vuelo_max_tarifa['numero_vuelo']} de ${vuelo_max_tarifa['tarifa']:,.2f} MXN es el más caro, "
          f"{vuelo_min_tarifa['numero_vuelo']} de ${vuelo_min_tarifa['tarifa']:,.2f} MXN es el más barato.")

    print("\n• Análisis de Pasajeros (Máximo y Mínimo):")
    print(f"{vuelo_max_pasajeros['numero_vuelo']} con {vuelo_max_pasajeros['pasajeros']} pasajeros es el que tiene más pasajeros, "
          f"{vuelo_min_pasajeros['numero_vuelo']} con {vuelo_min_pasajeros['pasajeros']} pasajeros es el que tiene menos pasajeros.")

# ==========================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ==========================================================================
```

FASE 5: Refinamiento y Comentarios

> *"Pedir a la IA que documente el código generado, explicando qué hace cada bloque clave mediante comentarios (#) pertinentes y concisos."*

A continuación se muestra la interacción con *Google Gemini* en forma de texto para la quinta parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo E se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT 5.1: *"A partir de este código actual para AeroRegistro, como parte final, documenta el código generado, explicando qué hace cada bloque clave mediante comentarios (#) claros, pertinentes y concisos, tiene que verse limpio, ordenado y fácil de comprender. No modifiques la funcionalidad ni lógica del programa, conserva las instrucciones, variables, estructuras de datos y cálculos existentes. Agrega comentarios al inicio de cada sección principal para identificar su propósito, documenta las instrucciones y métodos importantes, explicando brevemente qué función cumplen dentro del programa, describe brevemente los cálculos y agrega los comentarios adicionales que consideres necesarios para facilitar la comprensión del código."*

RESPUESTA:

Python

```
# ============================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# Propósito: Capturar e ingresar iterativamente los datos de los vuelos de AeroRegistro,
# validando que los datos numéricos sean positivos y estandarizando las cadenas a mayúsculas.
# ============================================================================

# Lista principal 'inventario' que almacena la colección de diccionarios (un diccionario por vuelo)
inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\n")

# Ciclo principal de captura continua de datos
while True:
    # Captura del identificador de vuelo:
    # - .strip() elimina espacios en blanco al inicio/final.
    # - .upper() convierte el texto ingresado a mayúsculas para estandarizar la salida.
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip().upper()

    # Condición de salida del ciclo: termina la captura si se ingresa '*' o se deja en blanco
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    # Lectura y estandarización a mayúsculas de origen, destino y aerolínea
    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip().upper()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip().upper()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip().upper()

    # Bucle de validación para el número de pasajeros (debe ser entero y mayor a 0)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positivo.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Bucle de validación para la tarifa del boleto (debe ser valor numérico flotante y mayor a 0)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Creación del diccionario de vuelo con los pares clave-valor de cada atributo
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    # Agregar el diccionario individual al inventario principal
    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante comprensión de conjuntos (Set Comprehension).
#    Garantiza la eliminación automática de valores duplicados.
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas inmutables mediante comprensión de listas (List Comprehension).
#    Crea una tupla de tres elementos (numero_vuelo, aerolinea, destino) por cada registro.
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# 3. Cálculos matemáticos y analítica de datos
# Cálculo de la cantidad general de registros capturados en la lista
total_vuelos = len(inventario)

if total_vuelos > 0:
    # Conteo de frecuencia y cálculo del número de aerolíneas distintas usando dict.get()
    conteo_aerolineas = {}
    for vuelo in inventario:
        aero = vuelo['aerolinea']
        conteo_aerolineas[aero] = conteo_aerolineas.get(aero, 0) + 1
    aerolineas_unicas = len(conteo_aerolineas)

    # Conteo de frecuencia por cada destino único registrado
    conteo_destinos = {}
    for vuelo in inventario:
        dest = vuelo['destino']
        conteo_destinos[dest] = conteo_destinos.get(dest, 0) + 1
    num_destinos_unicos = len(conteo_destinos)

    # Totales acumulados y cálculo del promedio de pasajeros
    total_pasajeros = sum(vuelo['pasajeros'] for vuelo in inventario)
    promedio_pasajeros = total_pasajeros / total_vuelos

    # Totales acumulados y cálculo del promedio de tarifas por vuelo
```

```
    total_tarifas = sum(vuelo['tarifa'] for vuelo in inventario)
    promedio_tarifa = total_tarifas / total_vuelos

    # Identificación del vuelo con la tarifa máxima y mínima utilizando max/min y funciones lambda como clave de evaluación
    vuelo_max_tarifa = max(inventario, key=lambda x: x['tarifa'])
    vuelo_min_tarifa = min(inventario, key=lambda x: x['tarifa'])

    # Identificación del vuelo con el mayor y menor número de pasajeros a bordo
    vuelo_max_pasajeros = max(inventario, key=lambda x: x['pasajeros'])
    vuelo_min_pasajeros = min(inventario, key=lambda x: x['pasajeros'])

# ============================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ============================================================================


# ============================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# Propósito: Formatear y proyectar visualmente en pantalla las lecturas de los datos
# crudos, conjuntos, tuplas, tabla formateada con alineación y reporte estadístico.
# ============================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión directa de la lista de diccionarios (datos sin procesar)
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Muestra de las colecciones derivadas (Set y Lista de Tuplas)
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
print(historial)

# 3. Presentación visual del inventario formateado en columnas tabulares
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    # Encabezados con formateo f-strings: < indica alineación izquierda y > alineación derecha con ancho especificado
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15} | {'PASAJEROS':<10} | {'TARIFA (MXN)':>12}"
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    # Recorrido del inventario formateando valores numéricos de tarifas a dos decimales (,.2f)
    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# 4. Impresión estructurada del análisis matemático y conteos analíticos
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    # Cantidad total de vuelos procesados
    print(f"• Total de vuelos registrados: {total_vuelos}\n")

    # Desglose numérico por cada aerolínea registrada
    print(f"• Cantidad de aerolíneas distintas: {aerolineas_unicas}")
    print("  Desglose por aerolínea:")
    for aerolinea, cantidad in conteo_aerolineas.items():
        print(f"    - {aerolinea}: {cantidad} vuelo(s)")

    # Desglose numérico por cada destino registrado
    print(f"\n• Número de destinos únicos: {num_destinos_unicos}")
    print("  Desglose por destino:")
    for destino, cantidad in conteo_destinos.items():
        print(f"    - {destino}: {cantidad} vuelo(s)")

    print("-" * 60)

    # Totales y promedios aplicados a la variable pasajeros
    print(f"• Total de pasajeros registrados:   {total_pasajeros:,} pasajeros")
    print(f"• Promedio de pasajeros por vuelo:  {promedio_pasajeros:.2f} pasajeros")
    print("-" * 60)

    # Totales y promedios aplicados al monto monetario de tarifas
    print(f"• Suma total de tarifas:            ${total_tarifas:,.2f} MXN")
    print(f"• Promedio de tarifa por vuelo:     ${promedio_tarifa:,.2f} MXN")
    print("-" * 60)

    # Enunciados descriptivos que destacan los valores extremos (Máximos y Mínimos)
    print("• Análisis de Tarifas (Máximo y Mínimo):")
    print(f"  {vuelo_max_tarifa['numero_vuelo']} de ${vuelo_max_tarifa['tarifa']:,.2f} MXN es el más caro, "
          f"{vuelo_min_tarifa['numero_vuelo']} de ${vuelo_min_tarifa['tarifa']:,.2f} MXN es el más barato.")
```

```
    print("\n• Análisis de Pasajeros (Máximo y Mínimo):")
    print(f"  {vuelo_max_pasajeros['numero_vuelo']} con {vuelo_max_pasajeros['pasajeros']} pasajeros es el que tiene más pasajeros, "
          f"{vuelo_min_pasajeros['numero_vuelo']} con {vuelo_min_pasajeros['pasajeros']} pasajeros es el que tiene menos pasajeros.")

# ==============================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ==============================================================================
```

4. Ejemplo de Ejecución:

A continuación se muestra el **resultado** al ejecutar el programa. En el Anexo F se muestra la evidencia de lo mostrado en la consola en forma de capturas de pantalla.

RESULTADO:

```
--- Sistema de Vuelos: AeroRegistro ---
Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.

Número de vuelo (ej. AA-101): am-202
Ciudad de origen (ej. Ciudad de México): cdmx
Ciudad de destino (ej. Guadalajara): monterrey
Aerolínea (ej. AeroMexico): aeromexico
Número de pasajeros (ej. 180): 150
Tarifa del boleto en pesos (ej. 2350.75): 3200.0
-> Vuelo guardado correctamente en el inventario.

Número de vuelo (ej. AA-101): vb-410
Ciudad de origen (ej. Ciudad de México): guadalajara
Ciudad de destino (ej. Guadalajara): cancun
Aerolínea (ej. AeroMexico): volaris
Número de pasajeros (ej. 180): 180
Tarifa del boleto en pesos (ej. 2350.75): 4500.5
-> Vuelo guardado correctamente en el inventario.

Número de vuelo (ej. AA-101): vb-311
Ciudad de origen (ej. Ciudad de México): cdmx
Ciudad de destino (ej. Guadalajara): tijuana
Aerolínea (ej. AeroMexico): volaris
Número de pasajeros (ej. 180): 120
Tarifa del boleto en pesos (ej. 2350.75): 2800.0
-> Vuelo guardado correctamente en el inventario.

Número de vuelo (ej. AA-101): ij-088
Ciudad de origen (ej. Ciudad de México): monterrey
Ciudad de destino (ej. Guadalajara): cdmx
Aerolínea (ej. AeroMexico): interjet
Número de pasajeros (ej. 180): 90
Tarifa del boleto en pesos (ej. 2350.75): 1950.75
-> Vuelo guardado correctamente en el inventario.

Número de vuelo (ej. AA-101):

================================================================================
                          REPORTES Y SALIDAS DE DATOS
================================================================================

--- 1. DATOS CRUDOS ---
Inventario completo (Lista de diccionarios):
[{'numero_vuelo': 'AM-202', 'origen': 'CDMX', 'destino': 'MONTERREY', 'aerolinea': 'AEROMEXICO',
'pasajeros': 150, 'tarifa': 3200.0}, {'numero_vuelo': 'VB-410', 'origen': 'GUADALAJARA', 'destino': 'CANCUN',
 'aerolinea': 'VOLARIS',   'pasajeros': 180, 'tarifa': 4500.5},  {'numero_vuelo': 'VB-311', 'origen': 'CDMX',
  'destino': 'TIJUANA', 'aerolinea': 'VOLARIS', 'pasajeros': 120, 'tarifa': 2800.0}, {'numero_vuelo': 'IJ-088',
   'origen': 'MONTERREY', 'destino': 'CDMX', 'aerolinea': 'INTERJET', 'pasajeros': 90, 'tarifa': 1950.75}]

--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---
Destinos únicos registrados (Conjunto):
{'CANCUN', 'TIJUANA', 'CDMX', 'MONTERREY'}

Historial inmutable de vuelos (Tuplas):
[('AM-202', 'AEROMEXICO', 'MONTERREY'), ('VB-410', 'VOLARIS', 'CANCUN'), ('VB-311', 'VOLARIS', 'TIJUANA'), ('IJ-088',
 'INTERJET', 'CDMX')]

--- 3. TABLA DE DATOS DE VUELOS ---
-------------------------------------------------------------------------------------------------
#VUELO     | ORIGEN               | DESTINO              | AEROLÍNEA       | PASAJEROS  | TARIFA (MXN)
-------------------------------------------------------------------------------------------------
AM-202     | CDMX                 | MONTERREY            | AEROMEXICO      | 150        | $   3,200.00
VB-410     | GUADALAJARA          | CANCUN               | VOLARIS         | 180        | $   4,500.50
VB-311     | CDMX                 | TIJUANA              | VOLARIS         | 120        | $   2,800.00
IJ-088     | MONTERREY            | CDMX                 | INTERJET        | 90         | $   1,950.75
-------------------------------------------------------------------------------------------------

--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---
• Total de vuelos registrados: 4

• Cantidad de aerolíneas distintas: 3
  Desglose por aerolínea:
    - AEROMEXICO: 1 vuelo(s)
    - VOLARIS: 2 vuelo(s)
    - INTERJET: 1 vuelo(s)

• Número de destinos únicos: 4
```

```
  Desglose por destino:
    - MONTERREY: 1 vuelo(s)
    - CANCUN: 1 vuelo(s)
    - TIJUANA: 1 vuelo(s)
    - CDMX: 1 vuelo(s)
------------------------------------------------------------
• Total de pasajeros registrados:   540 pasajeros
• Promedio de pasajeros por vuelo:  135.00 pasajeros
------------------------------------------------------------
• Suma total de tarifas:            $12,451.25 MXN
• Promedio de tarifa por vuelo:     $3,112.81 MXN
------------------------------------------------------------
• Análisis de Tarifas (Máximo y Mínimo):
  VB-410 de $4,500.50 MXN es el más caro, IJ-088 de $1,950.75 MXN es el más barato.

• Análisis de Pasajeros (Máximo y Mínimo):
  VB-410 con 180 pasajeros es el que tiene más pasajeros, IJ-088 con 90 pasajeros es el que tiene menos pasajeros.
```

5. Reflexión

Utilizar *Google Gemini* fue de ayuda para realizar el programa de forma progresiva. En general se realizaron cinco partes: estructura y captura; extracción de colecciones especiales; formateo y visualización; analítica y cálculos matemáticos; refinamiento y comentarios. Sin embargo, para algunas de ellas (1 y 4) se tuvo que agregar otro prompt debido a que el resultado inicial tenía algunos detalles como algún nombre o formato mostrado que se requería de diferente manera, por lo que al utilizar esta herramienta se tiene que ser muy específico con los prompts que se utilizan y se tiene que estar ejcutando el código para saber si es necesario modificar algo.

Anexo A: Evidencia de la FASE 1

Actúa como un experto en Python. Empieza a desarrollar un programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro. Solamente escribe la lógica (estructura básica) del ciclo while para capturar las entradas del usuario, a quien se le van a solicitar los siguientes datos: numero_vuelo (identificador del vuelo, ej. 'AA-101'), origen (ciudad de salida, ej. 'Ciudad de México'), destino (ciudad de llegada, ej. 'Guadalajara'), aerolinea (nombre de la aerolínea, ej. 'AeroMéxico', 'Volaris', 'Interjet'), pasajeros (número de pasajeros a bordo, ej. 180) y tarifa (precio del boleto en pesos, ej. 2350.75). La entrada de datos termina cuando el usuario ingresa el caracter *. Los datos de cada vuelo ingresado deben almacenarse en un diccionario y todos los diccionarios de vuelos se agregarán a una lista principal que representará el inventario (lista de diccionarios).

Python

```python
vuelos = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' en el número de vuelo para terminar la captura.\n")

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida
    if numero_vuelo == '*':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Conversión a entero y flotante para valores numéricos
    pasajeros = int(input("Número de pasajeros (ej. 180): "))
    tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))

    # Creación del diccionario por vuelo
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    # Agregar diccionario a la lista principal
    vuelos.append(vuelo)
    print("-> Vuelo guardado correctamente.\n")

print("\nCaptura finalizada. Inventario registrado:")
print(vuelos)
```

Modifica el nombre de la lista, actualmente se llama vuelos pero debe llamarse inventario. Agrega validación para los datos que lo requieren (pasajeros y tarifa, que además deben ser valores positivos). Agrega que la entrada, además de terminar al ingresar el caracter *, pueda terminar también cuando se ingresa un vuelo vacío.

Python

```python
inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positi
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

print("\nCaptura finalizada. Inventario registrado:")
print(inventario)
```

Anexo B: Evidencia de la FASE 2

Continuando con el desarrollo del programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro, y tomando en cuenta el código anterior. Ahora, recorre los datos capturados por el usuario, almacenados en inventario, y extrae un conjunto (set), llamado destinos_unicos, con los destinos únicos para garantizar que no haya ciudades repetidas. Además, genera una lista de tuplas, llamada historial, para un registro histórico inmutable, donde cada tupla contenga exclusivamente (numero_vuelo, aerolinea, destino). Agrega esto al código anterior, sin modificar lo que ya está.

Python

```python
inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positi
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")
```

Python

```
    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

print("\nCaptura finalizada. Inventario registrado:")
print(inventario)

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# Mostrar resultados
print("\n--- Resultados del Procesamiento ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
print(historial)
```

Anexo C: Evidencia de la FASE 3

Continuando con el desarrollo del programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro, y tomando en cuenta el código anterior, procede a la parte la implementación de las salidas en pantalla: impresión de los datos crudos, visualización de tuplas/sets y el formateo de una tabla tabular alineada para los vuelos. Agrega esto al código anterior, sin modificar lo que ya está. Marca claramente el inicio y fin de la sección de entrada, y haz lo mismo para el inicio de salida.

Python

```
# ================================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# ================================================================================

inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positi
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---
```

Python

```
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# ==========================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ==========================================================================



# ==========================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# ==========================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión de datos crudos
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
for item in historial:
    print(f"  Vuelo: {item[0]} | Aerolínea: {item[1]} | Destino: {item[2]}")

# 3. Formateo de tabla tabular alineada
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    # Encabezados de la tabla con f-strings y formato de ancho de columna
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15}
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    # Filas de la tabla alineadas
    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# ==========================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ==========================================================================
```

Anexo D: Evidencia de la FASE 4



Continuando con el desarrollo del programa para procesar el registro de vuelos de una aerolínea llamada AeroRegistro, y tomando en cuenta el código anterior, procede a la parte la analítica y de cálculos matemáticos, para agregar contadores (calcular el número total de vuelos registrados, la cantidad de aerolíneas diferentes y el número de destinos únicos.), sumar totales (calcular el total de pasajeros registrados y la suma de todas las tarifas de los vuelos), calcular promedios (obtener el promedio de pasajeros por vuelo y el promedio de las tarifas registradas) y encontrar los valores máximos y mínimos de los registros (identificar el mayor y el menor número de pasajeros registrados en un vuelo, así como la tarifa más alta y la más baja). Agrega esto al código anterior, sin modificar lo que ya está. Muestra en salida esta información, además agrega un apartado de cálculos.

Python

```python
# ================================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# ================================================================================

inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positi
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
```

Python

```
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# 3. Cálculos matemáticos y analítica
total_vuelos = len(inventario)

if total_vuelos > 0:
    # Contadores
    aerolineas_unicas = len({vuelo['aerolinea'] for vuelo in inventario})
    num_destinos_unicos = len(destinos_unicos)

    # Totales
    total_pasajeros = sum(vuelo['pasajeros'] for vuelo in inventario)
    total_tarifas = sum(vuelo['tarifa'] for vuelo in inventario)

    # Promedios
    promedio_pasajeros = total_pasajeros / total_vuelos
    promedio_tarifa = total_tarifas / total_vuelos

    # Máximos y Mínimos
    max_pasajeros = max(vuelo['pasajeros'] for vuelo in inventario)
    min_pasajeros = min(vuelo['pasajeros'] for vuelo in inventario)
    max_tarifa = max(vuelo['tarifa'] for vuelo in inventario)
    min_tarifa = min(vuelo['tarifa'] for vuelo in inventario)

# ==============================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ==============================================================================



# ==============================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# ==============================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión de datos crudos
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
for item in historial:
    print(f"  Vuelo: {item[0]} | Aerolínea: {item[1]} | Destino: {item[2]}")

# 3. Formateo de tabla tabular alineada
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    # Encabezados de la tabla con f-strings y formato de ancho de columna
```

Python

```
else:
    # Encabezados de la tabla con f-strings y formato de ancho de columna
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15}
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    # Filas de la tabla alineadas
    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# 4. Apartado de Cálculos y Analítica
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    print(f"• Total de vuelos registrados:      {total_vuelos}")
    print(f"• Cantidad de aerolíneas distintas:  {aerolineas_unicas}")
    print(f"• Número de destinos únicos:        {num_destinos_unicos}")
    print("-" * 50)
    print(f"• Total de pasajeros registrados:   {total_pasajeros:,} pasajeros")
    print(f"• Suma total de tarifas:            ${total_tarifas:,.2f} MXN")
    print("-" * 50)
    print(f"• Promedio de pasajeros por vuelo:  {promedio_pasajeros:.2f} pasajeros
    print(f"• Promedio de tarifa por vuelo:     ${promedio_tarifa:,.2f} MXN")
    print("-" * 50)
    print(f"• Pasajeros por vuelo (Máx / Mín):  {max_pasajeros} / {min_pasajeros}")
    print(f"• Tarifa por vuelo (Máx / Mín):     ${max_tarifa:,.2f} MXN / ${min_tari

# ============================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ============================================================================
```

A partir de este código generado para AeroRegistro, para refinar el programa realiza algunas modificaciones. En Historial inmutable de vuelos (Lista de Tuplas), escribe solamente la lista historial. En las aerolíneas distintas, además de poner el total, haz un desglose mostrando el nombre de las aerolíneas y cuantas veces se repite cada una. Para el número de destinos únicos, además de mostrar el total, haz un desglose donde muestres el nombre del destino y cuántas veces se repite. El totales y promedios haz la siguiente modificación: que se imprima el total de pasajeros y debajo el promedio de pasajeros por vuelo. Luego muestra la línea separadora para después mostrar la suma total de tarifas y debajo el promedio de tarifa por vuelo. Para máximos y mínimos, escribe explícitamente los enunciados, por ejemplo VB-410 de $4,500.50 es el más caro, IJ-088 de $1,950.75 es el más barato. Lo mismo para pasajeros. Además, para un mejor acomodo visual en la salida, implementa que todos los valores de entrada se muestren en mayúsculas.

y cuántas veces se repite. El totales y promedios haz la siguiente modificación: que se imprima el total de pasajeros y debajo el promedio de pasajeros por vuelo. Luego muestra la línea separadora para después mostrar la suma total de tarifas y debajo el promedio de tarifa por vuelo. Para máximos y mínimos, escribe explícitamente los enunciados, por ejemplo VB-410 de $4,500.50 es el más caro, IJ-088 de $1,950.75 es el más barato. Lo mismo para pasajeros. Además, para un mejor acomodo visual en la salida, implementa que todos los valores de entrada se muestren en mayúsculas.

Mostrar código <>

Python

```python
# ============================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# ============================================================================

inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\

while True:
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip().upper()

    # Condición de salida: caracter '*' o entrada vacía
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip().upper()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip().upper()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip().upper()

    # Validación para número de pasajeros (entero positivo)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positi
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Validación para tarifa (flotante positivo)
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Registro del vuelo en la lista principal 'inventario'
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }
```

Python

```
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante un set
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas para historial inmutable
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# 3. Cálculos matemáticos y analítica
total_vuelos = len(inventario)

if total_vuelos > 0:
    # Desglose y conteo de aerolíneas
    conteo_aerolineas = {}
    for vuelo in inventario:
        aero = vuelo['aerolinea']
        conteo_aerolineas[aero] = conteo_aerolineas.get(aero, 0) + 1
    aerolineas_unicas = len(conteo_aerolineas)

    # Desglose y conteo de destinos
    conteo_destinos = {}
    for vuelo in inventario:
        dest = vuelo['destino']
        conteo_destinos[dest] = conteo_destinos.get(dest, 0) + 1
    num_destinos_unicos = len(conteo_destinos)

    # Totales y Promedios
    total_pasajeros = sum(vuelo['pasajeros'] for vuelo in inventario)
    promedio_pasajeros = total_pasajeros / total_vuelos

    total_tarifas = sum(vuelo['tarifa'] for vuelo in inventario)
    promedio_tarifa = total_tarifas / total_vuelos

    # Vuelos con valores Máximos y Mínimos
    vuelo_max_tarifa = max(inventario, key=lambda x: x['tarifa'])
    vuelo_min_tarifa = min(inventario, key=lambda x: x['tarifa'])

    vuelo_max_pasajeros = max(inventario, key=lambda x: x['pasajeros'])
    vuelo_min_pasajeros = min(inventario, key=lambda x: x['pasajeros'])

# ================================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ================================================================================


# ================================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# ================================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión de datos crudos
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
```

Python

```python
# 2. Visualización de Sets y Tuplas
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
print(historial)

# 3. Formateo de tabla tabular alineada
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15}
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# 4. Apartado de Cálculos y Analítica
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    print(f"• Total de vuelos registrados: {total_vuelos}\n")

    # Desglose de aerolíneas
    print(f"• Cantidad de aerolíneas distintas: {aerolineas_unicas}")
    print("  Desglose por aerolínea:")
    for aerolinea, cantidad in conteo_aerolineas.items():
        print(f"    - {aerolinea}: {cantidad} vuelo(s)")

    # Desglose de destinos
    print(f"\n• Número de destinos únicos: {num_destinos_unicos}")
    print("  Desglose por destino:")
    for destino, cantidad in conteo_destinos.items():
        print(f"    - {destino}: {cantidad} vuelo(s)")

    print("-" * 60)

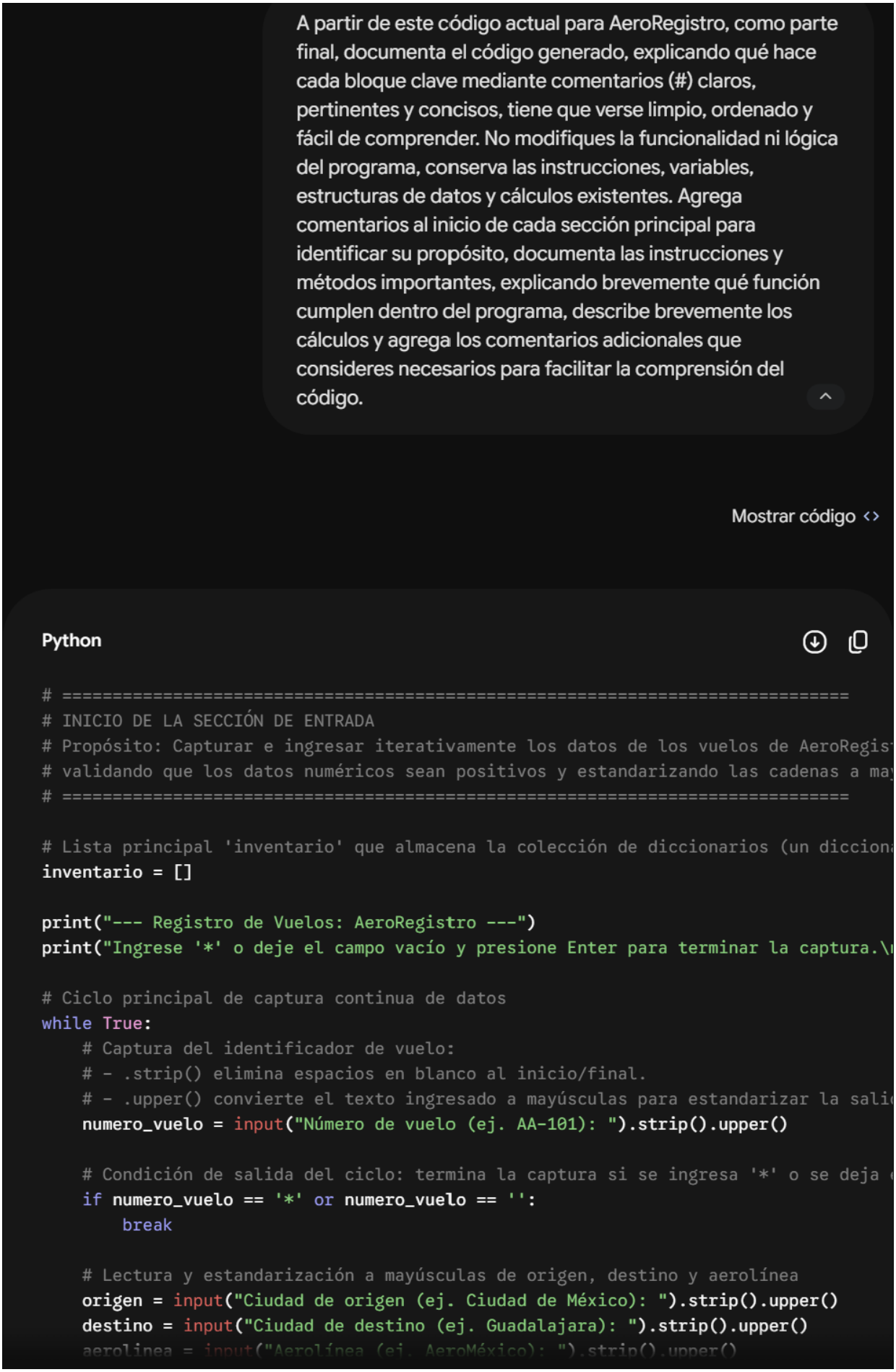
    # Totales y Promedios agrupados
    print(f"• Total de pasajeros registrados:   {total_pasajeros:,} pasajeros")
    print(f"• Promedio de pasajeros por vuelo:  {promedio_pasajeros:.2f} pasajeros")
    print("-" * 60)
    print(f"• Suma total de tarifas:            ${total_tarifas:,.2f} MXN")
    print(f"• Promedio de tarifa por vuelo:     ${promedio_tarifa:,.2f} MXN")
    print("-" * 60)

    # Enunciados explícitos de Máximos y Mínimos
    print("• Análisis de Tarifas (Máximo y Mínimo):")
    print(f"  {vuelo_max_tarifa['numero_vuelo']} de ${vuelo_max_tarifa['tarifa']:,.2
          f"{vuelo_min_tarifa['numero_vuelo']} de ${vuelo_min_tarifa['tarifa']:,.2f}

    print("\n• Análisis de Pasajeros (Máximo y Mínimo):")
    print(f"  {vuelo_max_pasajeros['numero_vuelo']} con {vuelo_max_pasajeros['pasaje
          f"{vuelo_min_pasajeros['numero_vuelo']} con {vuelo_min_pasajeros['pasajero

# ==========================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ==========================================================================
```

Anexo E: Evidencia de la FASE 5



A partir de este código actual para AeroRegistro, como parte final, documenta el código generado, explicando qué hace cada bloque clave mediante comentarios (#) claros, pertinentes y concisos, tiene que verse limpio, ordenado y fácil de comprender. No modifiques la funcionalidad ni lógica del programa, conserva las instrucciones, variables, estructuras de datos y cálculos existentes. Agrega comentarios al inicio de cada sección principal para identificar su propósito, documenta las instrucciones y métodos importantes, explicando brevemente qué función cumplen dentro del programa, describe brevemente los cálculos y agrega los comentarios adicionales que consideres necesarios para facilitar la comprensión del código.

Mostrar código

Python

```python
# ================================================================================
# INICIO DE LA SECCIÓN DE ENTRADA
# Propósito: Capturar e ingresar iterativamente los datos de los vuelos de AeroRegis
# validando que los datos numéricos sean positivos y estandarizando las cadenas a ma
# ================================================================================

# Lista principal 'inventario' que almacena la colección de diccionarios (un diccion
inventario = []

print("--- Registro de Vuelos: AeroRegistro ---")
print("Ingrese '*' o deje el campo vacío y presione Enter para terminar la captura.\

# Ciclo principal de captura continua de datos
while True:
    # Captura del identificador de vuelo:
    # - .strip() elimina espacios en blanco al inicio/final.
    # - .upper() convierte el texto ingresado a mayúsculas para estandarizar la sali
    numero_vuelo = input("Número de vuelo (ej. AA-101): ").strip().upper()

    # Condición de salida del ciclo: termina la captura si se ingresa '*' o se deja
    if numero_vuelo == '*' or numero_vuelo == '':
        break

    # Lectura y estandarización a mayúsculas de origen, destino y aerolínea
    origen = input("Ciudad de origen (ej. Ciudad de México): ").strip().upper()
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip().upper()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip().upper()
```

Python

```
    destino = input("Ciudad de destino (ej. Guadalajara): ").strip().upper()
    aerolinea = input("Aerolínea (ej. AeroMéxico): ").strip().upper()

    # Bucle de validación para el número de pasajeros (debe ser entero y mayor a 0)
    while True:
        try:
            pasajeros = int(input("Número de pasajeros (ej. 180): "))
            if pasajeros > 0:
                break
            print("  [Error] El número de pasajeros debe ser un número entero positi
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un número entero.")

    # Bucle de validación para la tarifa del boleto (debe ser valor numérico flotant
    while True:
        try:
            tarifa = float(input("Tarifa del boleto en pesos (ej. 2350.75): "))
            if tarifa > 0:
                break
            print("  [Error] La tarifa debe ser un valor positivo superior a 0.")
        except ValueError:
            print("  [Error] Entrada inválida. Ingrese un valor numérico adecuado.")

    # Creación del diccionario de vuelo con los pares clave-valor de cada atributo
    vuelo = {
        'numero_vuelo': numero_vuelo,
        'origen': origen,
        'destino': destino,
        'aerolinea': aerolinea,
        'pasajeros': pasajeros,
        'tarifa': tarifa
    }

    # Agregar el diccionario individual al inventario principal
    inventario.append(vuelo)
    print("-> Vuelo guardado correctamente en el inventario.\n")

# --- PROCESAMIENTO DE DATOS DEL INVENTARIO ---

# 1. Extracción de destinos únicos mediante comprensión de conjuntos (Set Comprehens
#    Garantiza la eliminación automática de valores duplicados.
destinos_unicos = {vuelo['destino'] for vuelo in inventario}

# 2. Generación de lista de tuplas inmutables mediante comprensión de listas (List C
#    Crea una tupla de tres elementos (numero_vuelo, aerolinea, destino) por cada re
historial = [
    (vuelo['numero_vuelo'], vuelo['aerolinea'], vuelo['destino'])
    for vuelo in inventario
]

# 3. Cálculos matemáticos y analítica de datos
# Cálculo de la cantidad general de registros capturados en la lista
total_vuelos = len(inventario)
```

Python

```python
# Cálculo de la cantidad general de registros capturados en la lista
total_vuelos = len(inventario)

if total_vuelos > 0:
    # Conteo de frecuencia y cálculo del número de aerolíneas distintas usando dict.
    conteo_aerolineas = {}
    for vuelo in inventario:
        aero = vuelo['aerolinea']
        conteo_aerolineas[aero] = conteo_aerolineas.get(aero, 0) + 1
    aerolineas_unicas = len(conteo_aerolineas)

    # Conteo de frecuencia por cada destino único registrado
    conteo_destinos = {}
    for vuelo in inventario:
        dest = vuelo['destino']
        conteo_destinos[dest] = conteo_destinos.get(dest, 0) + 1
    num_destinos_unicos = len(conteo_destinos)

    # Totales acumulados y cálculo del promedio de pasajeros
    total_pasajeros = sum(vuelo['pasajeros'] for vuelo in inventario)
    promedio_pasajeros = total_pasajeros / total_vuelos

    # Totales acumulados y cálculo del promedio de tarifas por vuelo
    total_tarifas = sum(vuelo['tarifa'] for vuelo in inventario)
    promedio_tarifa = total_tarifas / total_vuelos

    # Identificación del vuelo con la tarifa máxima y mínima utilizando max/min y fu
    vuelo_max_tarifa = max(inventario, key=lambda x: x['tarifa'])
    vuelo_min_tarifa = min(inventario, key=lambda x: x['tarifa'])

    # Identificación del vuelo con el mayor y menor número de pasajeros a bordo
    vuelo_max_pasajeros = max(inventario, key=lambda x: x['pasajeros'])
    vuelo_min_pasajeros = min(inventario, key=lambda x: x['pasajeros'])

# ================================================================================
# FIN DE LA SECCIÓN DE ENTRADA
# ================================================================================



# ================================================================================
# INICIO DE LA SECCIÓN DE SALIDA
# Propósito: Formatear y proyectar visualmente en pantalla las lecturas de los datos
# crudos, conjuntos, tuplas, tabla formateada con alineación y reporte estadístico.
# ================================================================================

print("\n" + "="*80)
print("                          REPORTES Y SALIDAS DE DATOS")
print("="*80)

# 1. Impresión directa de la lista de diccionarios (datos sin procesar)
print("\n--- 1. DATOS CRUDOS ---")
print("Inventario completo (Lista de Diccionarios):")
print(inventario)
```

Python

```python
print("Inventario completo (Lista de Diccionarios):")
print(inventario)

# 2. Muestra de las colecciones derivadas (Set y Lista de Tuplas)
print("\n--- 2. ESTRUCTURAS DE DATOS DERIVADAS ---")
print("Destinos únicos registrados (Set):")
print(destinos_unicos)

print("\nHistorial inmutable de vuelos (Lista de Tuplas):")
print(historial)

# 3. Presentación visual del inventario formateado en columnas tabulares
print("\n--- 3. TABULAR DE INVENTARIO DE VUELOS ---")
if not inventario:
    print("No se registraron vuelos en el inventario.")
else:
    # Encabezados con formateo f-strings: < indica alineación izquierda y > alineaci
    header = f"{'VUELO':<10} | {'ORIGEN':<20} | {'DESTINO':<20} | {'AEROLÍNEA':<15}
    print("-" * len(header))
    print(header)
    print("-" * len(header))

    # Recorrido del inventario formateando valores numéricos de tarifas a dos decima
    for vuelo in inventario:
        print(
            f"{vuelo['numero_vuelo']:<10} | "
            f"{vuelo['origen']:<20} | "
            f"{vuelo['destino']:<20} | "
            f"{vuelo['aerolinea']:<15} | "
            f"{vuelo['pasajeros']:<10} | "
            f"${vuelo['tarifa']:>11,.2f}"
        )
    print("-" * len(header))

# 4. Impresión estructurada del análisis matemático y conteos analíticos
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    # Cantidad total de vuelos procesados
    print(f"• Total de vuelos registrados: {total_vuelos}\n")

    # Desglose numérico por cada aerolínea registrada
    print(f"• Cantidad de aerolíneas distintas: {aerolineas_unicas}")
    print("  Desglose por aerolínea:")
    for aerolinea, cantidad in conteo_aerolineas.items():
        print(f"    - {aerolinea}: {cantidad} vuelo(s)")

    # Desglose numérico por cada destino registrado
    print(f"\n• Número de destinos únicos: {num_destinos_unicos}")
    print("  Desglose por destino:")
    for destino, cantidad in conteo_destinos.items():
```

Python

```
        )
    print("-" * len(header))

# 4. Impresión estructurada del análisis matemático y conteos analíticos
print("\n--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---")
if total_vuelos == 0:
    print("No hay datos suficientes para realizar cálculos analíticos.")
else:
    # Cantidad total de vuelos procesados
    print(f"• Total de vuelos registrados: {total_vuelos}\n")

    # Desglose numérico por cada aerolínea registrada
    print(f"• Cantidad de aerolíneas distintas: {aerolineas_unicas}")
    print("  Desglose por aerolínea:")
    for aerolinea, cantidad in conteo_aerolineas.items():
        print(f"    - {aerolinea}: {cantidad} vuelo(s)")

    # Desglose numérico por cada destino registrado
    print(f"\n• Número de destinos únicos: {num_destinos_unicos}")
    print("  Desglose por destino:")
    for destino, cantidad in conteo_destinos.items():
        print(f"    - {destino}: {cantidad} vuelo(s)")

    print("-" * 60)

    # Totales y promedios aplicados a la variable pasajeros
    print(f"• Total de pasajeros registrados:   {total_pasajeros:,} pasajeros")
    print(f"• Promedio de pasajeros por vuelo:  {promedio_pasajeros:.2f} pasajeros")
    print("-" * 60)

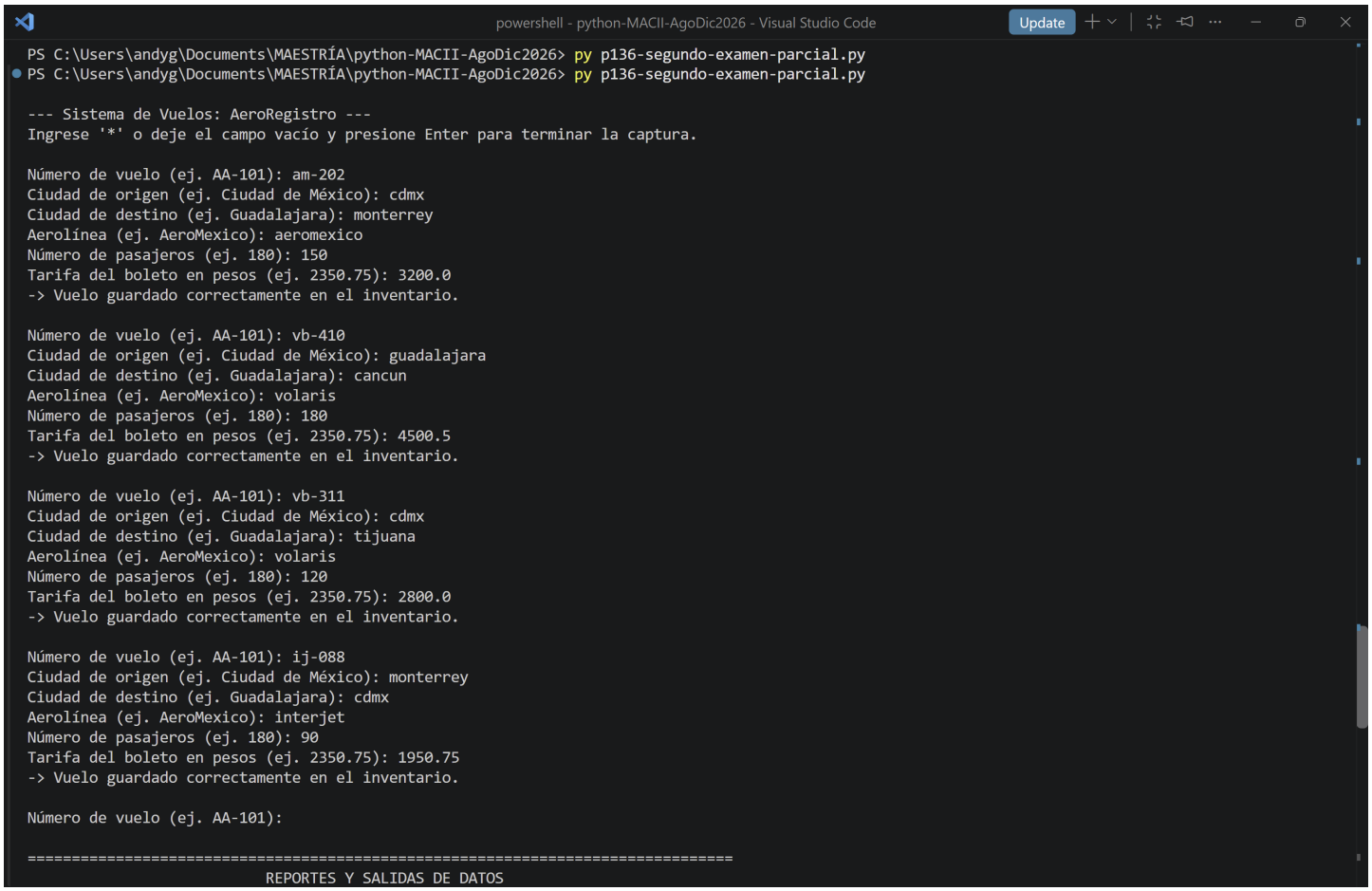
    # Totales y promedios aplicados al monto monetario de tarifas
    print(f"• Suma total de tarifas:            ${total_tarifas:,.2f} MXN")
    print(f"• Promedio de tarifa por vuelo:     ${promedio_tarifa:,.2f} MXN")
    print("-" * 60)

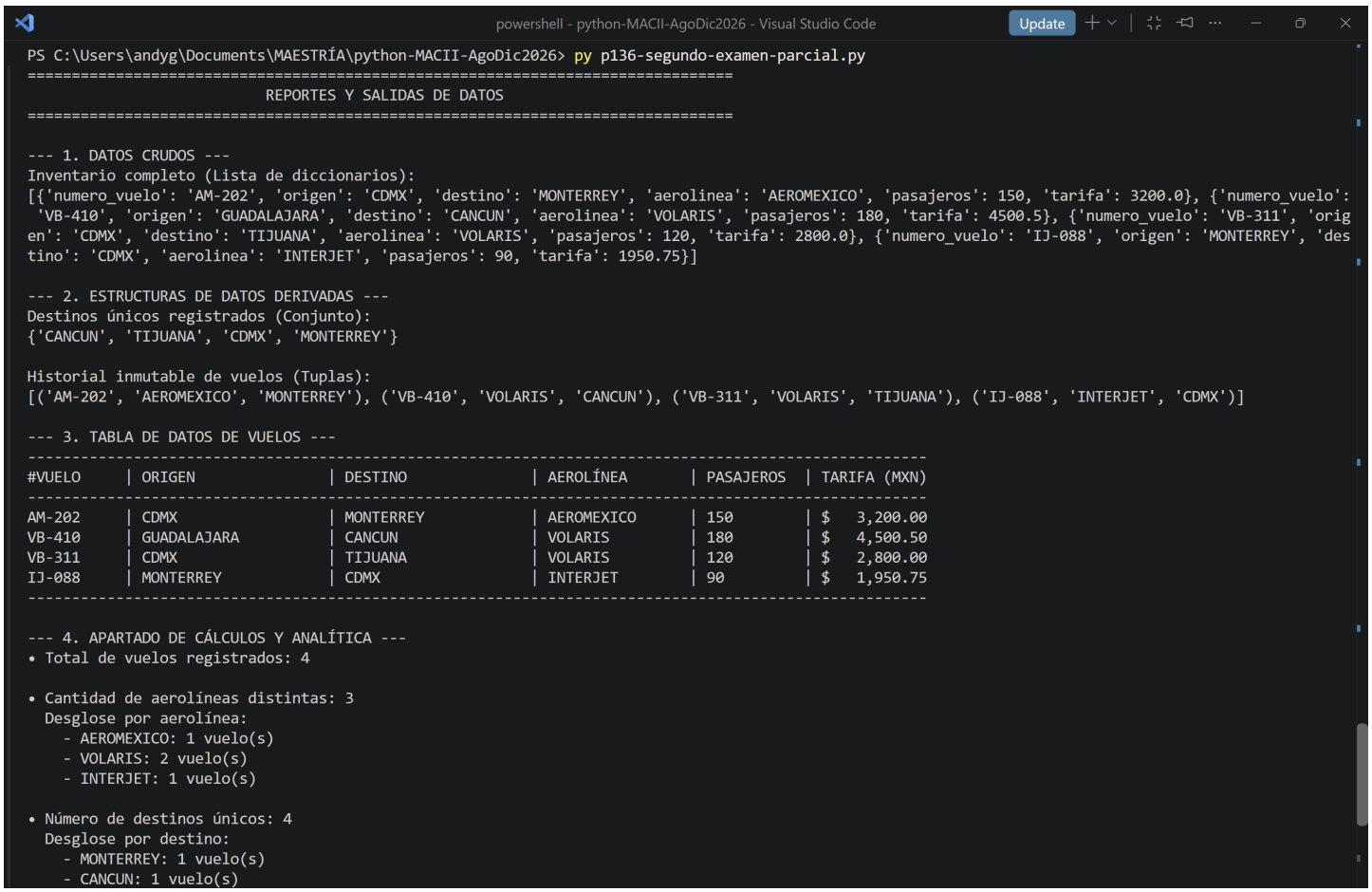
    # Enunciados descriptivos que destacan los valores extremos (Máximos y Mínimos)
    print("• Análisis de Tarifas (Máximo y Mínimo):")
    print(f"  {vuelo_max_tarifa['numero_vuelo']} de ${vuelo_max_tarifa['tarifa']:,.2
          f"{vuelo_min_tarifa['numero_vuelo']} de ${vuelo_min_tarifa['tarifa']:,.2f}

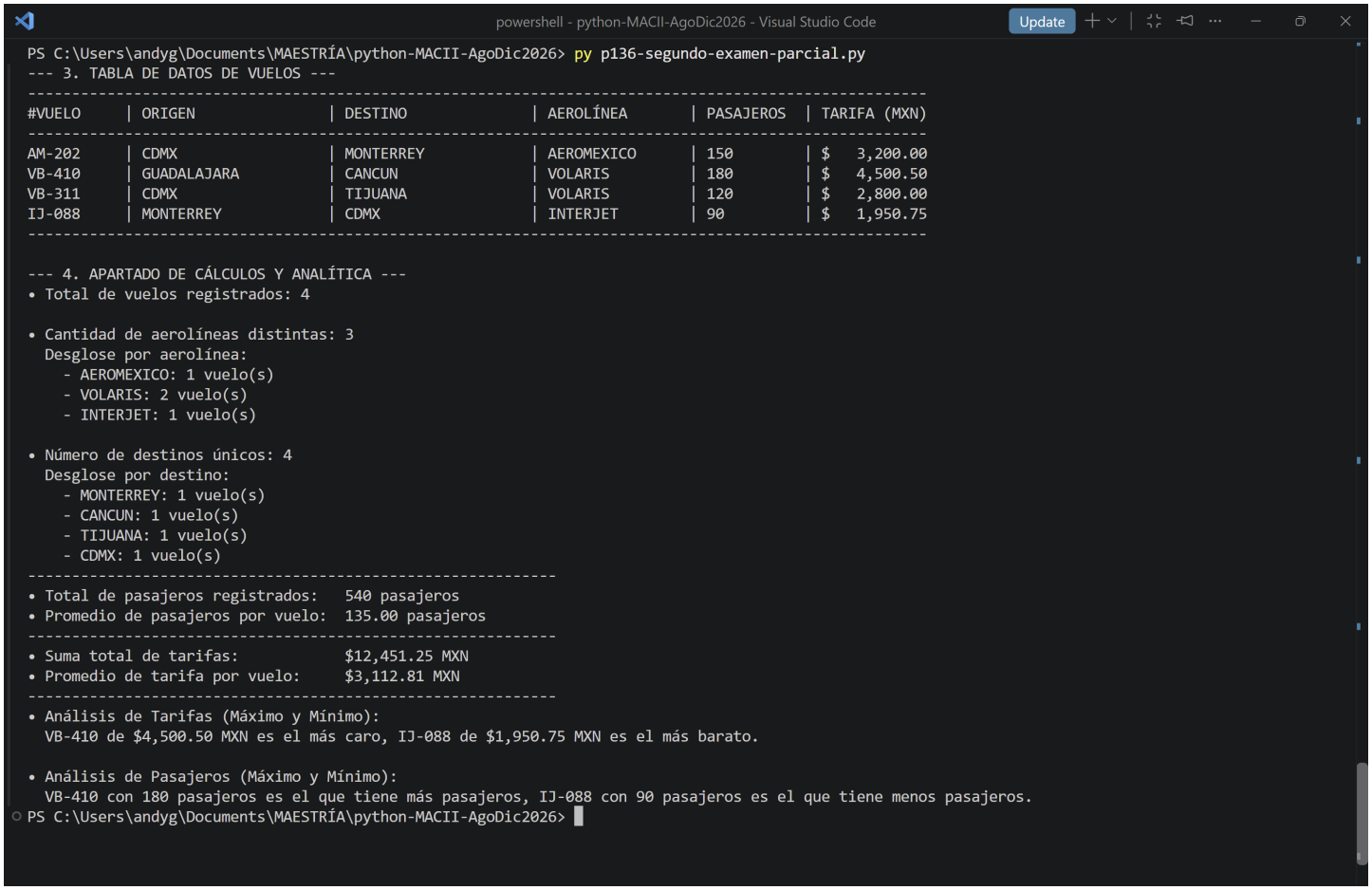
    print("\n• Análisis de Pasajeros (Máximo y Mínimo):")
    print(f"  {vuelo_max_pasajeros['numero_vuelo']} con {vuelo_max_pasajeros['pasaje
          f"{vuelo_min_pasajeros['numero_vuelo']} con {vuelo_min_pasajeros['pasajero

# ================================================================================
# FIN DE LA SECCIÓN DE SALIDA
# ================================================================================
```

Anexo F: Evidencia de Ejecución






powershell - python-MACII-AgoDic2026 - Visual Studio Code
Update
PS C:\Users\andyg\Documents\MAESTRÍA\python-MACII-AgoDic2026> py p136-segundo-examen-parcial.py
--- 3. TABLA DE DATOS DE VUELOS ---
#VUELO | ORIGEN | DESTINO | AEROLÍNEA | PASAJEROS | TARIFA (MXN)
AM-202 | CDMX | MONTERREY | AEROMEXICO | 150 | $ 3,200.00
VB-410 | GUADALAJARA | CANCUN | VOLARIS | 180 | $ 4,500.50
VB-311 | CDMX | TIJUANA | VOLARIS | 120 | $ 2,800.00
IJ-088 | MONTERREY | CDMX | INTERJET | 90 | $ 1,950.75
--- 4. APARTADO DE CÁLCULOS Y ANALÍTICA ---
• Total de vuelos registrados: 4
• Cantidad de aerolíneas distintas: 3
Desglose por aerolínea:
- AEROMEXICO: 1 vuelo(s)
- VOLARIS: 2 vuelo(s)
- INTERJET: 1 vuelo(s)
• Número de destinos únicos: 4
Desglose por destino:
- MONTERREY: 1 vuelo(s)
- CANCUN: 1 vuelo(s)
- TIJUANA: 1 vuelo(s)
- CDMX: 1 vuelo(s)
• Total de pasajeros registrados: 540 pasajeros
• Promedio de pasajeros por vuelo: 135.00 pasajeros
• Suma total de tarifas: $12,451.25 MXN
• Promedio de tarifa por vuelo: $3,112.81 MXN
• Análisis de Tarifas (Máximo y Mínimo):
VB-410 de $4,500.50 MXN es el más caro, IJ-088 de $1,950.75 MXN es el más barato.
• Análisis de Pasajeros (Máximo y Mínimo):
VB-410 con 180 pasajeros es el que tiene más pasajeros, IJ-088 con 90 pasajeros es el que tiene menos pasajeros.
PS C:\Users\andyg\Documents\MAESTRÍA\python-MACII-AgoDic2026>